

EVIDENCIAS ACADÉMICAS · SEMANA 3 · ACTIVIDAD 1

Robustez con encapsulamiento, herencia y polimorfismo

Quantum Core — Diseño de los pilares de la Programación Orientada a Objetos

Evidencia técnica basada en el código y la ejecución real del proyecto

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software
ACTIVIDAD	Semana 3 · Actividad 1 — Pilares de la POO
PROYECTO	Quantum Core
ARCHIVO PRINCIPAL	diseno_pilares_poo.py
DATOS DE PRUEBA	20 transacciones simuladas · 19 válidas, 1 rechazada
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	3 de septiembre de 2026

Objetivo, requisitos demostrados y reglas de impacto

La actividad transforma la solución orientada a objetos de la Semana 2 en un diseño más seguro y extensible.

Se aplicó **encapsulamiento** para controlar el monto, **herencia** para reutilizar la estructura común y **polimorfismo** para calcular un impacto distinto según el tipo de transacción. La evidencia se basa en el código y la ejecución real del proyecto ubicado en `ProyectosCeipa/Semana_3`.

Requisitos demostrados

REQUISITO	IMPLEMENTACIÓN	EVIDENCIA
Encapsulamiento	Getter, setter y atributo <code>_monto</code>	Figura 1 · página 4
Herencia	<code>TransaccionBase</code> y cuatro clases hijas	Figura 2 · página 5
Polimorfismo	<code>calcular_impacto()</code> sobrescrito en cada hija	Figuras 2 y 5
Robustez	<code>try / except</code> durante la lectura	Figura 4 · página 7
Prueba de validación	Rechazo del registro T005 con monto negativo	Figura 7 · página 10

Reglas de impacto por tipo

TIPO	CLASE	REGLA APLICADA	IMPACTO SOBRE 100.000
Crédito	<code>TransaccionCredito</code>	2 % del monto	2.000
Débito	<code>TransaccionDebito</code>	Comisión fija	1.500
Efectivo	<code>TransaccionEfectivo</code>	1 % del monto	1.000
Cripto	<code>TransaccionCripto</code>	3 % del monto	3.000

Secuencia del desarrollo

Ocho pasos ordenados, desde la preparación de la carpeta hasta la construcción del conjunto de pruebas.

#	ETAPA	DESCRIPCIÓN
1	Preparación	Se creó la carpeta Semana_3, se copió el archivo de la Semana 2 y se renombró como <code>diseño_pilares_poo.py</code> .
2	Encapsulamiento	El monto se almacena en <code>_monto</code> y se controla mediante <code>@property</code> y <code>@monto.setter</code> .
3	Validación	El setter convierte el dato a <code>float</code> y genera <code>ValueError</code> cuando el monto es negativo.
4	Herencia	<code>TransaccionCredito</code> , <code>TransaccionDebito</code> , <code>TransaccionEfectivo</code> y <code>TransaccionCripto</code> reutilizan <code>TransaccionBase</code> .
5	Polimorfismo	Cada clase hija sobrescribe <code>calcular_impacto()</code> con una fórmula propia.
6	Creación por tipo	<code>crear_transaccion()</code> normaliza el texto y selecciona la clase correspondiente.
7	Lectura robusta	<code>enumerate()</code> identifica la línea y <code>try / except</code> permite continuar después de un dato inválido.
8	Pruebas	Se conservaron los seis datos del profesor y se añadieron catorce casos para completar veinte registros.

FUENTE DE LA ACTIVIDAD

<https://fundamentos-de-software.onrender.com/semanas/3>

Clase base, getter, setter y validación del monto

Código real de TransaccionBase. Se observa el constructor, el *getter*, el *setter*, la validación del monto y los métodos comunes que heredan todas las clases hijas.

```

25 # 1. CLASE BASE: DATOS Y COMPORTAMIENTOS COMUNES
26 # =====
27 class TransaccionBase:
28     """
29     Representa la estructura común de todas las transacciones.
30
31     El constructor, la validación del monto y los métodos de consulta se
32     escriben una sola vez en esta clase. Las clases hijas los reutilizan y
33     solamente implementan la regla que las hace diferentes.
34     """
35
36     def __init__(self, cliente_id, tipo, monto):
37         """
38         Inicializa una transacción.
39
40         Parámetros:
41             cliente_id: identificador del registro, por ejemplo T001.
42             tipo: nombre del tipo de transacción.
43             monto: valor monetario de la transacción.
44
45         La asignación self.monto = monto pasa por el setter. De esta manera,
46         incluso el valor inicial se valida antes de almacenarse.
47         """
48         self.cliente_id = cliente_id
49         self.tipo = tipo
50         self.monto = monto
51
52     # Getter: permite consultar el monto mediante objeto.monto. El valor real
53     # permanece almacenado internamente en el atributo protegido _monto.
54     @property
55     def monto(self):
56         """Devuelve el monto protegido de la transacción."""
57         return self._monto
58
59     # Setter: se ejecuta al crear la transacción o al asignar un nuevo monto.
60     # Convierte el dato a float y evita guardar cantidades negativas.
61     @monto.setter
62     def monto(self, nuevo_monto):
63         """Valida y almacena un nuevo monto."""
64         nuevo_monto = float(nuevo_monto)
65
66         if nuevo_monto < 0:
67             raise ValueError("El monto asignado no puede ser negativo.")
68
69         # Se usa _monto para almacenar el dato sin llamar otra vez al setter.
70         self._monto = nuevo_monto
71
72     def calcular_impacto(self):
73         """
74         Declara el comportamiento que debe implementar cada clase hija.
75
76         La clase base no posee una fórmula general. Si una hija olvida crear
77         su propia versión, este error informa claramente qué falta.
78         """
79         raise NotImplementedError(
80             "Cada tipo de transacción debe calcular su propio impacto."
81         )
82
83     def obtener_informacion(self):
84         """Devuelve los datos principales en un texto fácil de leer."""
85         return (
86             f"ID: {self.cliente_id} | "
87             f"Tipo: {self.tipo} | "
88             f"Monto: ${self.monto:.2f}"
89         )
90
91     def obtener_monto(self):
92         """Devuelve el monto para operaciones como el cálculo del total."""
93         return self.monto
94
95     def es_del_tipo(self, tipo_filtro):
96         """Indica si la transacción corresponde al tipo solicitado."""

```

Figura 1. Clase TransaccionBase: constructor, @property, @monto.setter y validación.

Las cuatro clases hijas

Las cuatro clases heredan de `TransaccionBase` y sobrescriben `calcular_impacto()` con reglas diferentes. Esa respuesta distinta al mismo método es el polimorfismo.

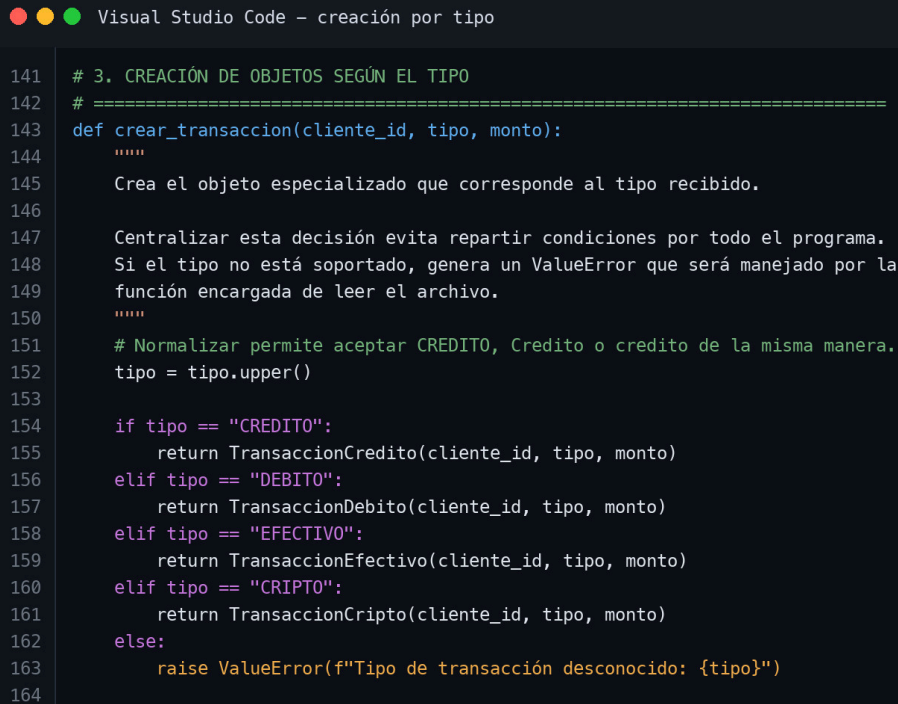
```
Visual Studio Code – herencia y polimorfismo

101 # 2. CLASES HIJAS: HERENCIA Y POLIMORFISMO
102 # =====
103 # Todas las hijas reciben el constructor, el getter, el setter y los métodos de
104 # consulta de TransaccionBase. Cada una sobrescribe calcular_impacto() con una
105 # regla propia; esa respuesta diferente al mismo método es el polimorfismo.
106
107
108 class TransaccionCredito(TransaccionBase):
109     """Representa un crédito cuyo impacto corresponde al 2 % del monto."""
110
111     def calcular_impacto(self):
112         """Calcula un impacto porcentual del 2 %."""
113         return self.monto * 0.02
114
115
116 class TransaccionDebito(TransaccionBase):
117     """Representa un débito con una comisión fija de 1.500."""
118
119     def calcular_impacto(self):
120         """Devuelve la comisión fija de una transacción débito."""
121         return 1500
122
123
124 class TransaccionEfectivo(TransaccionBase):
125     """Representa una operación en efectivo con impacto del 1 %."""
126
127     def calcular_impacto(self):
128         """Calcula un impacto porcentual del 1 %."""
129         return self.monto * 0.01
130
131
132 class TransaccionCripto(TransaccionBase):
133     """Representa una operación con criptomonedas e impacto del 3 %."""
134
135     def calcular_impacto(self):
136         """Calcula un impacto porcentual del 3 %."""
137         return self.monto * 0.03
```

Figura 2. Clases hijas y sus versiones de `calcular_impacto()`.

Función crear_transaccion()

La función reconoce Crédito, Débito, Efectivo y Cripto. El tipo se normaliza a mayúsculas con `upper()` antes de comparar, de modo que acepta `CREDITO`, `Credito` o `credito` indistintamente.



```

141 # 3. CREACIÓN DE OBJETOS SEGÚN EL TIPO
142 # =====
143 def crear_transaccion(cliente_id, tipo, monto):
144     """
145     Crea el objeto especializado que corresponde al tipo recibido.
146
147     Centralizar esta decisión evita repartir condiciones por todo el programa.
148     Si el tipo no está soportado, genera un ValueError que será manejado por la
149     función encargada de leer el archivo.
150     """
151     # Normalizar permite aceptar CREDITO, Credito o credito de la misma manera.
152     tipo = tipo.upper()
153
154     if tipo == "CREDITO":
155         return TransaccionCredito(cliente_id, tipo, monto)
156     elif tipo == "DEBITO":
157         return TransaccionDebito(cliente_id, tipo, monto)
158     elif tipo == "EFECTIVO":
159         return TransaccionEfectivo(cliente_id, tipo, monto)
160     elif tipo == "CRIPTO":
161         return TransaccionCripto(cliente_id, tipo, monto)
162     else:
163         raise ValueError(f"Tipo de transacción desconocido: {tipo}")
164
  
```

Figura 3. `crear_transaccion()`: selección de la clase según el tipo recibido.

MANEJO DEL CASO NO SOPORTADO

Si el tipo no corresponde a ninguna de las cuatro clases, la función genera un `ValueError` que será capturado por la función de lectura. Centralizar esta decisión en un solo lugar evita repartir condiciones por todo el programa.

Manejo de errores durante la carga

La lectura enumera las líneas, crea los objetos y reporta los datos inválidos sin detener el procesamiento del resto del archivo.

```

167 # 4. LECTURA Y CONVERSIÓN DEL ARCHIVO A OBJETOS
168 # =====
169 def leer_y_almacenar_datos(nombre_archivo):
170     """
171     Lee el archivo y convierte cada registro válido en un objeto.
172
173     Cada línea debe contener cliente_id, tipo y monto separados por comas. El
174     resultado es una lista que puede mezclar objetos de todas las clases hijas.
175     Los registros con valores inválidos se reportan y se ignoran sin detener la
176     lectura de las líneas siguientes.
177     """
178     lista_transacciones = []
179
180     # El bloque with cierra automáticamente el archivo cuando termina la lectura.
181     with open(nombre_archivo, "r", encoding="utf-8") as archivo:
182         # enumerate proporciona el número real de cada línea para los avisos.
183         for numero_linea, linea in enumerate(archivo, start=1):
184             # strip elimina espacios externos y split separa los tres campos.
185             datos = linea.strip().split(",")
186
187             # Solo se procesa una línea cuando contiene exactamente tres datos.
188             if len(datos) == 3:
189                 try:
190                     transaccion = crear_transaccion(
191                         datos[0].strip(),
192                         datos[1].strip(),
193                         float(datos[2].strip()),
194                     )
195
196                     # La lista recibe únicamente objetos creados correctamente.
197                     lista_transacciones.append(transaccion)
198
199                 except ValueError as error:
200                     # Se identifica el registro y la línea sin detener el programa.
201                     print(
202                         f"Registro {datos[0].strip()} ignorado "
203                         f"en la línea {numero_linea}: {error}"
204                     )
205
206     return lista_transacciones
207
208

```

Figura 4. leer_y_almacenar_datos() con enumerate() y bloque try / except.

Totalización, filtrado y demostración del polimorfismo

Se conserva `filtrar_por_tipo()` de la Semana 2, se calcula el monto total y el ciclo principal recorre todos los objetos llamando al mismo método.

```

Visual Studio Code – consultas y ejecución principal

210 # 5. FUNCIONES DE CONSULTA CONSERVADAS DE LA SEMANA 2
211 # =====
212 def calcular_monto_total(lista_transacciones):
213     """Suma los montos de todos los objetos recibidos."""
214     total_monto = 0.0
215
216     for transaccion in lista_transacciones:
217         total_monto = total_monto + transaccion.obtener_monto()
218
219     return total_monto
220
221
222 def filtrar_por_tipo(lista_transacciones, tipo_filtro):
223     """
224     Devuelve una lista con las transacciones del tipo solicitado.
225
226     Esta función se conserva porque sigue siendo útil para consultar grupos
227     concretos, aunque la demostración principal recorra todos los objetos.
228     """
229     lista_filtrada = []
230
231     for transaccion in lista_transacciones:
232         if transaccion.es_del_tipo(tipo_filtro):
233             lista_filtrada.append(transaccion)
234
235     return lista_filtrada
236
237
238 # =====
239 # 6. FLUJO PRINCIPAL Y DEMOSTRACIÓN DEL POLIMORFISMO
240 # =====
241 def ejecutar_sistema():
242     """
243     Lee las transacciones, calcula el total y muestra el impacto de cada una.
244
245     El ciclo final no pregunta qué clase tiene cada objeto. Siempre ejecuta
246     calcular_impacto(), y Python selecciona la versión correspondiente.
247     """
248     transacciones = leer_y_almacenar_datos("transacciones.txt")
249     monto_total = calcular_monto_total(transacciones)
250
251     print(f'Monto total: ${monto_total:.2f}')
252     print("\n--- Transacciones válidas e impacto ---")
253
254     # Misma llamada, resultados distintos: aquí se observa el polimorfismo.
255     for transaccion in transacciones:
256         print(
257             transaccion.obtener_informacion(),
258             f"| Impacto: ${transaccion.calcular_impacto():.2f}",
259         )
260
261
262 # Este bloque evita que la demostración se ejecute al importar las clases desde
263 # otro archivo. Solo inicia el sistema cuando ejecutamos este archivo directamente.
264 if __name__ == "__main__":
265     ejecutar_sistema()

```

Figura 5. Funciones de consulta y `ejecutar_sistema()`, donde se observa el polimorfismo.

Archivo transacciones.txt con veinte registros

El conjunto conserva los seis datos entregados por el docente y añade catorce casos propios. El registro T005 mantiene deliberadamente un monto negativo para verificar el encapsulamiento.

```

Visual Studio Code - transacciones.txt

1 T001,CREDITO,500000
2 T002,DEBITO,80000
3 T003,CREDITO,1200000
4 T004,DEBITO,45000
5 T005,CREDITO,-1
6 T006,EFFECTIVO,10000
7 T007,CRIPTO,250000
8 T008,CREDITO,0
9 T009,DEBITO,150000
10 T010,EFFECTIVO,75000
11 T011,CRIPTO,980000
12 T012,CREDITO,350000
13 T013,DEBITO,220000
14 T014,EFFECTIVO,50000
15 T015,CRIPTO,125000
16 T016,CREDITO,2000000
17 T017,DEBITO,60000
18 T018,EFFECTIVO,310000
19 T019,CRIPTO,450000
20 T020,CREDITO,725000
  
```

Figura 6. Contenido de transacciones.txt con los veinte registros de prueba.

ID	TIPO	MONTO	ID	TIPO	MONTO
T001	CREDITO	500.000	T011	CRIPTO	980.000
T002	DEBITO	80.000	T012	CREDITO	350.000
T003	CREDITO	1.200.000	T013	DEBITO	220.000
T004	DEBITO	45.000	T014	EFFECTIVO	50.000
T005	CREDITO	-1	T015	CRIPTO	125.000
T006	EFFECTIVO	10.000	T016	CREDITO	2.000.000
T007	CRIPTO	250.000	T017	DEBITO	60.000
T008	CREDITO	0	T018	EFFECTIVO	310.000
T009	DEBITO	150.000	T019	CRIPTO	450.000
T010	EFFECTIVO	75.000	T020	CREDITO	725.000

Salida real de python3 diseno_pilares_poo.py

El programa reporta el registro rechazado con su número de línea, continúa con los diecinueve válidos y muestra el impacto calculado para cada uno.

```
Terminal - python3 diseno_pilares_poo.py

Registro T005 ignorado en la línea 5: El monto asignado no puede ser negativo.
Monto total: $7580000.00

--- Transacciones válidas e impacto ---
ID: T001 | Tipo: CREDITO | Monto: $500000.00 | Impacto: $10000.00
ID: T002 | Tipo: DEBITO | Monto: $80000.00 | Impacto: $1500.00
ID: T003 | Tipo: CREDITO | Monto: $1200000.00 | Impacto: $24000.00
ID: T004 | Tipo: DEBITO | Monto: $45000.00 | Impacto: $1500.00
ID: T006 | Tipo: EFECTIVO | Monto: $10000.00 | Impacto: $100.00
ID: T007 | Tipo: CRIPTO | Monto: $250000.00 | Impacto: $7500.00
ID: T008 | Tipo: CREDITO | Monto: $0.00 | Impacto: $0.00
ID: T009 | Tipo: DEBITO | Monto: $150000.00 | Impacto: $1500.00
ID: T010 | Tipo: EFECTIVO | Monto: $75000.00 | Impacto: $750.00
ID: T011 | Tipo: CRIPTO | Monto: $980000.00 | Impacto: $29400.00
ID: T012 | Tipo: CREDITO | Monto: $350000.00 | Impacto: $7000.00
ID: T013 | Tipo: DEBITO | Monto: $220000.00 | Impacto: $1500.00
ID: T014 | Tipo: EFECTIVO | Monto: $50000.00 | Impacto: $500.00
ID: T015 | Tipo: CRIPTO | Monto: $125000.00 | Impacto: $3750.00
ID: T016 | Tipo: CREDITO | Monto: $2000000.00 | Impacto: $40000.00
ID: T017 | Tipo: DEBITO | Monto: $60000.00 | Impacto: $1500.00
ID: T018 | Tipo: EFECTIVO | Monto: $310000.00 | Impacto: $3100.00
ID: T019 | Tipo: CRIPTO | Monto: $450000.00 | Impacto: $13500.00
ID: T020 | Tipo: CREDITO | Monto: $725000.00 | Impacto: $14500.00

Proceso finalizado con código de salida: 0
```

Figura 7. Ejecución completa: rechazo de T005, monto total e impacto por transacción.

PUNTOS CLAVE DE LA SALIDA

```
Registro T005 ignorado en la línea 5: El monto asignado no puede ser negativo.
Monto total: $7580000.00
...
Proceso finalizado con código de salida: 0
```

Resultados verificados y conclusión

Comparación entre lo solicitado por la actividad y lo observado en la ejecución real.

INDICADOR	RESULTADO VERIFICADO
Registros de entrada	20
Registros válidos	19
Registros rechazados	1 (T005, línea 5)
Monto total válido	\$7.580.000,00
Tipos procesados	Crédito, Débito, Efectivo y Cripto
Código de salida	0 — ejecución correcta

Distribución por tipo

TIPO	REGISTROS	MONTO	IMPACTO CALCULADO
Crédito	6	\$4.775.000	\$95.500
Débito	5	\$555.000	\$7.500
Efectivo	4	\$445.000	\$4.450
Cripto	4	\$1.805.000	\$54.150
Total	19	\$7.580.000	\$161.600

Conclusión

La ejecución confirmó que el `setter` impide almacenar montos negativos: el registro T005 fue identificado con su número de línea y las diecinueve transacciones válidas continuaron su procesamiento. El encapsulamiento concentra la validación dentro del objeto; la herencia evita repetir el constructor y los métodos comunes; y el polimorfismo permite usar `calcular_impacto()` sin preguntar qué clase posee cada transacción. La incorporación del tipo Cripto y el conjunto de veinte datos demuestran que la solución puede ampliarse y probarse con diferentes condiciones.